Module 8: Trees and Binary Trees

C Programming + Interview Preparation + LeetCode + LinkedIn Learning Path

---

Module Overview

Module	Trees and Binary Trees
Duration	5 Hours
Language	C
Difficulty	Beginner → Intermediate
Modules	Theory + Dry Run + Live Coding + LeetCode Practice

Learning Outcomes

After completing this module, learners will be able to:

- Understand the fundamentals of Trees.
- Differentiate between Linear and Non-Linear Data Structures.
- Explain Tree Terminology.
- Create Binary Trees using C.
- Perform Tree Traversals.
- Implement Binary Search Trees (BST).
- Perform Searching, Insertion, and Deletion in BST.
- Understand AVL Trees (Introduction).
- Solve beginner-level LeetCode Tree problems.
- Understand real-world applications of Trees.

Session Plan (5 Hours)

Topic	Hands-on
Introduction to Trees & Terminology	Tree Creation
Binary Trees	Node Creation
Binary Search Trees	Insert/Search/Delete
Tree Traversals	DFS Traversals
AVL Trees + Applications + LeetCode	Practice Problems

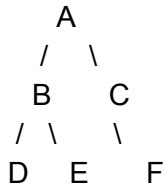
Module 1 – Introduction to Trees

What is a Tree?

A Tree is a **non-linear data structure** consisting of nodes connected using edges.

Unlike arrays and linked lists, trees organize data hierarchically.

Example



Why Trees?

Trees are used when

- Fast Searching
- Hierarchical Data
- File Systems
- Database Indexing
- Expression Evaluation

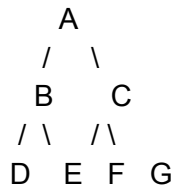
- Routing Algorithms

Linear vs Non-Linear Data Structures

Linear	Non-Linear
Array	Tree
Linked List	Graph
Stack	Heap
Queue	Trie

Tree Terminology

Example



Term	Meaning	Example
Root	Top node	A
Parent	Node having child	B
Child	Connected below parent	D
Leaf Node	No children	D,E,F,G
Internal Node	Has children	A,B,C
Edge	Connection	A-B
Height	Longest path	3

Depth	Distance from root	D = 2
Sibling	Same Parent	B,C
Subtree	Smaller Tree	Tree rooted at B

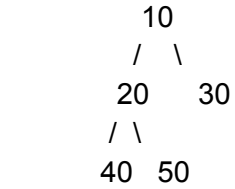
Binary Tree

Definition

A Binary Tree is a Tree in which every node has at most **two children**.

- Left Child
- Right Child

Example



Types of Binary Trees

Type	Description
Full Binary Tree	Every node has 0 or 2 children
Complete Binary Tree	Filled from left to right
Perfect Binary Tree	All leaves at same level
Balanced Tree	Height difference ≤1
Degenerate Tree	Like Linked List

Binary Tree Node in C

```
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

Creating a Node

Students will implement

- malloc()
- Pointer
- Structure

Binary Search Tree (BST)

Definition

A Binary Search Tree follows

Left Subtree

Smaller Values

Root

Current Node

Right Subtree

Greater Values

Example

```
        50
      /    \
```

```
    30      70
   /  \    /  \
  20   40 60   80
```

BST Operations

Students will implement

- Insert Node
- Search Node
- Delete Node
- Find Minimum
- Find Maximum

Complexity

Operation	Average	Worst
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Tree Traversals

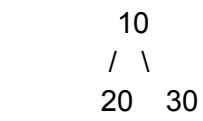
Preorder

Root

Left

Right

Example



Traversal

10 20 30

Inorder

Left

Root

Right

Example

20 10 30

Special Property

BST Inorder Traversal always gives

Sorted Order

Postorder

Left

Right

Root

Example

20 30 10

Traversal Visualization

```
      50
    /    \
   30     70
  /  \   /  \
 20  40 60  80
```

Traversal	Output
Preorder	50 30 20 40 70 60 80
Inorder	20 30 40 50 60 70 80
Postorder	20 40 30 60 80 70 50

AVL Trees (Introduction)

AVL Tree is a **Self-Balancing Binary Search Tree**.

Balance Factor

Height(Left) - Height(Right)

Must be

-1

0

+1

Whenever balance becomes

<-1

or

>1

Rotation is performed.

Types of Rotations

- Left Rotation
- Right Rotation
- Left Right Rotation
- Right Left Rotation

AVL Advantages

- Faster Search
- Balanced Height
- O(log n) Searching
- Used in Databases

Applications of Trees

Application	Tree Used
File Explorer	Tree
Windows Registry	Tree
XML Parser	Tree
HTML DOM	Tree
Compiler	Syntax Tree
Expression Evaluation	Binary Tree
Database Index	B Tree / B+ Tree
Routing Tables	Trie
Auto Complete	Trie
AI Decision Making	Decision Tree

Heap Sort	Heap
Priority Queue	Heap

Lab Activities

Students will implement

1. Create Tree
2. Insert Node
3. Search Node
4. Delete Node
5. Count Nodes
6. Find Height
7. Find Leaf Nodes
8. Mirror Tree
9. Preorder Traversal
10. Inorder Traversal
11. Postorder Traversal

LeetCode Practice (Recommended Progression)

S.No	LeetCode No.	Problem	Difficulty	Concept
1	94	Binary Tree Inorder Traversal	Easy	Inorder
2	144	Binary Tree Preorder Traversal	Easy	Preorder
3	145	Binary Tree Postorder Traversal	Easy	Postorder
4	100	Same Tree	Easy	Tree Comparison
5	101	Symmetric Tree	Easy	Recursion
6	104	Maximum Depth of Binary Tree	Easy	Height

7	226	Invert Binary Tree	Easy	Tree Manipulation
8	700	Search in a BST	Easy	BST Search
9	701	Insert into a BST	Medium	BST Insertion
10	98	Validate Binary Search Tree	Medium	BST Validation
11	530	Minimum Absolute Difference in BST	Easy	Inorder Property
12	230	Kth Smallest Element in a BST	Medium	Inorder + BST

LinkedIn Learning Path

Order	Topic	Search Keyword
1	Data Structures Foundations	Data Structures
2	Binary Trees	Binary Trees
3	Binary Search Trees	Binary Search Trees
4	Recursion	Recursion in C
5	Algorithms	Algorithms Foundations
6	Tree Traversals	Tree Traversal
7	AVL Trees	Self Balancing Trees
8	Data Structures Interview	Coding Interview: Data Structures
9	Problem Solving	Programming Foundations: Algorithms
10	DSA Interview Prep	Data Structures Interview Practice

Expected Coding Exercises

1. Create a Binary Tree using structures.
2. Insert nodes into a BST.

3. Search for a key in a BST.
4. Delete a node from a BST.
5. Count total nodes.
6. Count leaf nodes.
7. Find the height of a tree.
8. Perform Preorder traversal.
9. Perform Inorder traversal.
10. Perform Postorder traversal.
11. Mirror a Binary Tree.
12. Validate whether a tree satisfies BST properties.

---

94. Binary Tree Inorder Traversal

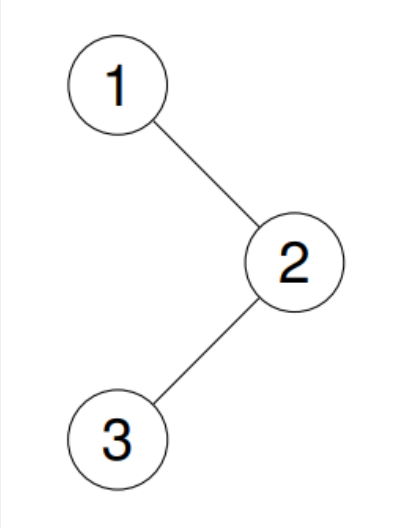
Given the root of a binary tree, return *the inorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

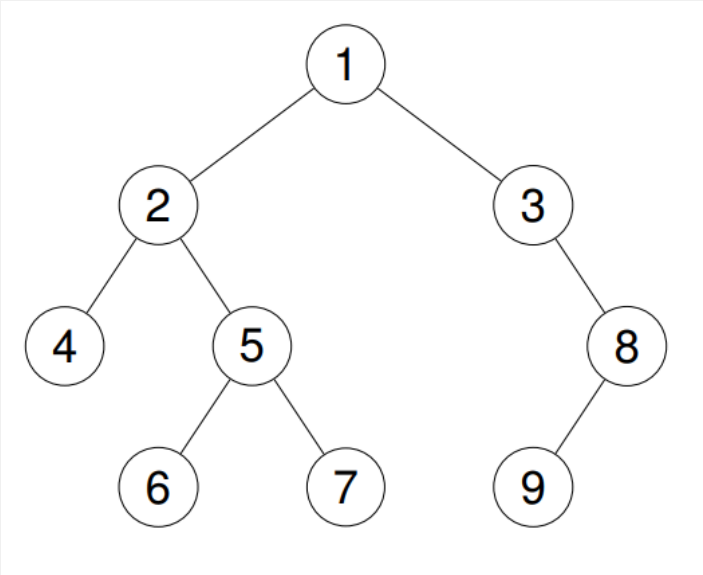


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- -100 <= Node.val <= 100

Follow up: Recursive solution is trivial, could you do it iteratively?

Start
  |
Is Node NULL?
  |

```
Yes --------> Return
  |
 No
  |
Traverse Left
  |
Visit Root
  |
Traverse Right
  |
Return
```

Steps :

Approach 1: Recursive Solution

Step 1 Start from the root node.

Step 2 If the current node is NULL,

- Return immediately.

Step 3 Recursively traverse the left subtree.

Step 4 Visit the current node.

- Store its value in the result array/list.

Step 5 Recursively traverse the right subtree.

Step 6 Continue until every node has been visited.

Step 7 Return the result list.

Approach 2: Iterative Solution (Using Stack)

Instead of recursion, maintain your own stack.

Step 1 Create an empty stack.

Step 2 Create an empty result list.

Step 3 Set current = root.

Step 4 Repeat while

- current is not NULL
  OR
- stack is not empty.

Step 5 Move to the leftmost node.

- Push every node into the stack.
- Move current = current->left.

Step 6 When current becomes NULL:

- Pop the top node from the stack.
- Visit the node.
- Store its value.

	Step 7 Move to the right child. • current = current->right **Step 8** Repeat Steps 5–7 until the stack becomes empty. **Step 9** Return the result list.

```c
//binary tree demo
//step1: we are going to create a binary tree and perform inorder traversal on it
//step2: we will create a structure for the binary tree node, which will have data, left and right pointers
//step3: we will create a function to create a new node, which will take an integer value as input and return a pointer to the new node
//step4: we will create a function to perform inorder traversal on the binary tree, which will take a pointer to the root node as input and print the data of each node in inorder sequence
//step5: we will create a main function to create a binary tree and call the inorder traversal function to print the data of each node in inorder sequence


#include <stdio.h>
#include <stdlib.h>

// Structure for Binary Tree Node
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};

// Function to create a new node
struct Node* createNode(int value)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node)); //
```

```
dynamically creating a new node

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Inorder Traversal (Left -> Root -> Right)
void inorderTraversal(struct Node *root)
{
    if(root == NULL)
        return;

    inorderTraversal(root->left); // passing the left child of the root node
to the function

    printf("%d ", root->data);

    inorderTraversal(root->right); // passing the right child of the root
node to the function
}

int main()
{
    /*
            1
             \
              2
             /
            3
    */

    struct Node *root = createNode(1);// creating the root node of the binary
tree
    root->right = createNode(2); // creating the right child of the root node
    root->right->left = createNode(3); // creating the left child of the
right child of the root node
```

```
    printf("Inorder Traversal: ");

    inorderTraversal(root); // calling the inorder traversal function and
passing the root node to it

    return 0;
}
```

Program Flow

Start
  |
Create Binary Tree
  |
Call inorderTraversal(root)
  |
Go Left
  |
Visit Root (Print)
  |
Go Right
  |
Repeat Until Tree Ends
  |
Stop

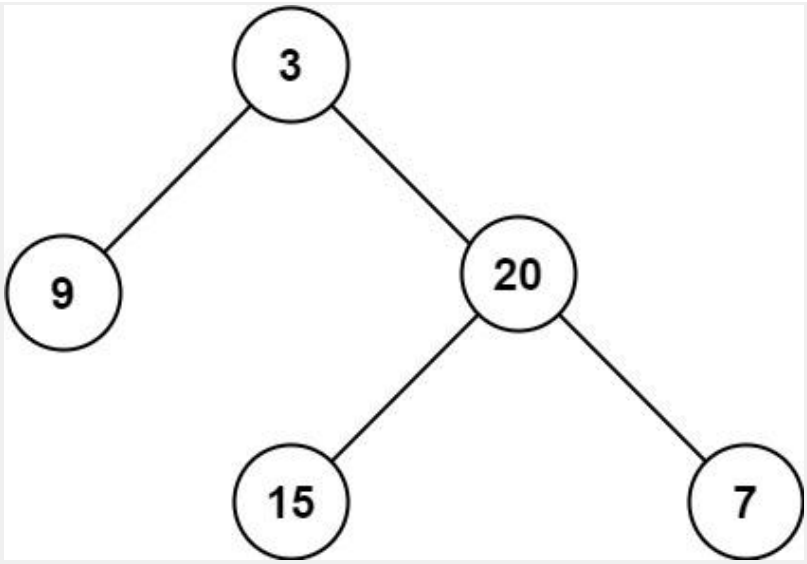
---

104. Maximum Depth of Binary Tree

Given the root of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Constraints:

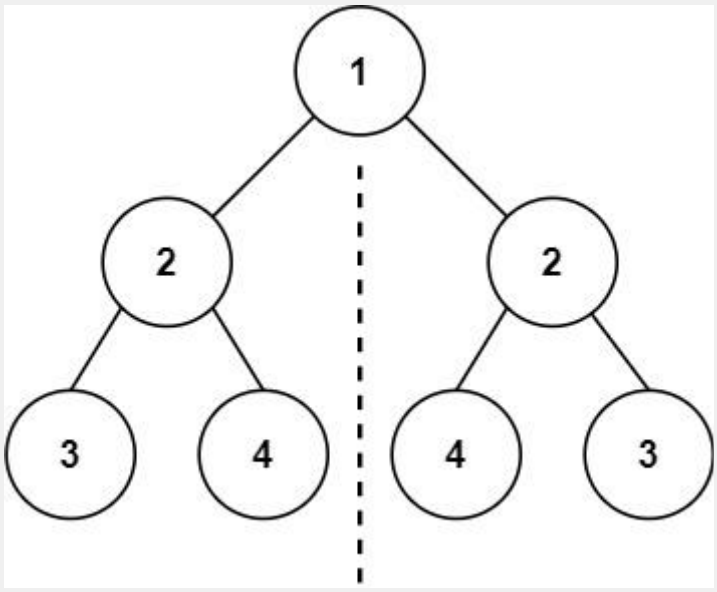
- The number of nodes in the tree is in the range [0, 10^4].
- -100 <= Node.val <= 100

```
Algorithm: MaximumDepth(root)

If root is NULL
    Return 0

leftDepth = MaximumDepth(root.left)

rightDepth = MaximumDepth(root.right)

Return 1 + Maximum(leftDepth, rightDepth)
```

101. Symmetric Tree

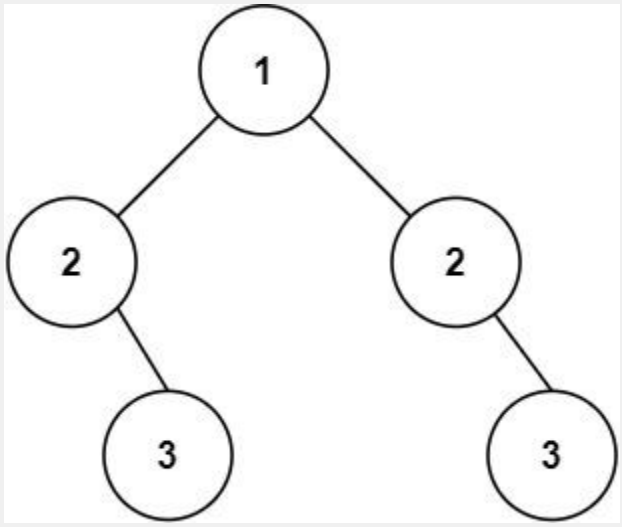
Given the root of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

Example 1:



Input: root = [1,2,2,3,4,4,3]
Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]
Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- -100 <= Node.val <= 100